



Introducción a JAVASCRIPT

(parte 2)

Contenido



- ❖ JSON.
- ❖ POO.
- ❖ Métodos de Arrays (map, filter y reduce).



JSON (1/4)

JSON (**J**ava**S**cript **O**bject **N**otation) es un formato de intercambio de datos ligero basado en un subconjunto de la sintaxis de JavaScript:

- ❖ literales de *arrays*.
- ❖ literales de *objetos*.

JSON es completamente independiente del lenguaje, pero utiliza las convenciones que son familiares para los programadores de la familia del lenguaje **C**, incluyendo **C++**, **C#**, **Java**, **Perl**, **Python** y muchos otros.



JSON (2/4)

Los objetos literales son la estructura de datos más usada en JavaScript, está compuesta de pares ordenados y agrupados en **claves** y **valores**.

- ❖ Se denomina **objeto literal** al objeto cuyas propiedades están declaradas *textualmente* en el código.
- ❖ Los objetos pueden almacenar métodos en su interior.
- ❖ Para acceder a las propiedades de un objeto dentro de un método es necesario usar la palabra reservada **this** (por scope de las variables).
- ❖ Se pueden añadir, actualizar o eliminar propiedades de una manera muy simple.



JSON (3/4)

- ❖ Un objeto comienza con { y termina con }.
- ❖ Cada propiedad es seguido por : y luego su correspondiente valor.
- ❖ Los pares clave/valor están separados por una , (coma).

```
const obj = { nombre : "Juan", edad : 25 };
```

- ❖ El método **JSON.parse()** analiza una cadena de texto como JSON, transformando dicha cadena en un objeto.

```
const obj = JSON.parse('{ "nombre" : "Pedro", "edad" : 56 }');
```



JSON (4/4)

- ❖ ***JSON.stringify()*** convierte un objeto a a notación JSON:
 - Las propiedades de objetos no-array no se garantiza que se realicen en un orden particular.
 - Los objetos Boolean, Number y String se convierten a sus valores primitivos.
 - Si se encuentra ***undefined*** durante la conversión o se omite (cuando se encuentra en un objeto) o se sustituye por ***null*** (cuando se encuentra en un array).

```
let objCadena = JSON.stringify(obj);
```



Ejemplos



POO en JavaScript

- ❖ Estructura básica de una clase.
- ❖ Accesores get y set.
- ❖ Herencia.
- ❖ Interfaces y clases abstractas.



Clases

Los miembros de una clase poseen modificadores de visibilidad:

- ❖ `public` (defecto), `protected` y `private`.
- ❖ Los miembros de una clase pueden ser estáticos y no estáticos.
- ❖ Con el modificador *static*, se marcan los miembros estáticos de una clase.
- ❖ Los constructores se definen utilizando la palabra reservada **constructor**.
- ❖ En JavaScript no existe la herencia múltiple.



```
class Usuario {  
  
    nombre; // público  
    #dni; // protegido  
    _edad; // privado  
  
    constructor(nombre, dni) {  
        this.nombre = nombre;  
        this.#dni = dni;  
        this._edad = 0;  
    }  
  
    get edad() {  
        return this._edad;  
    }  
  
    set edad(value) {  
        if (value.length > 0) {  
            console.log("Ingrese edad mayor a cero.");  
            return;  
        }  
        this._edad = value;  
    }  
  
    get dni(){  
        return this.#dni;  
    }  
  
    saludar(){  
        console.log(`Hola ${this.nombre}!!! tu dni es: ${this.#dni} y tu edad: ${this._edad}.`);  
    }  
}
```



Herencia (2/3)

```
class Vehiculo {  
    #marca;  
  
    constructor(marca){  
        this.#marca = marca;  
    }  
  
    toString() {  
        return `MARCA: ${this.#marca}`;  
    }  
}
```

```
class Auto extends Vehiculo{  
  
    color;  
  
    constructor(marca, color) {  
        super(marca);  
        this.color = color;  
    }  
  
    toString() {  
        return `${super.toString()} - COLOR: ${this.color}`;  
    }  
}
```



Clases abstractas (2/3)

- ❖ JavaScript no posee clases abstractas nativas, pero se pueden evitar crear instancias directas utilizando:

```
class Abstracta {  
    constructor() {  
        if(this.constructor === Abstracta) {  
            throw new Error("No se puede instanciar una clase abstracta.");  
        }  
    }  
}
```

```
class Concreta extends Abstracta {  
    constructor(){  
        super();  
    }  
}
```



Ejemplos



Métodos de Array

- ❖ Map.
- ❖ Filter.
- ❖ Reduce.



map (1/3)

La función **map** se utiliza cuando se necesita modificar cada elemento de un array de la misma forma.

Por ejemplo:

eleva al cuadrado cada elemento de un array de números, aplicar el IVA a un listado de precios o ejecutar una expresión regular a un array de Strings.

Cuando se llama a **map**, este ejecuta un Callback en cada elemento dentro del array, retornando un nuevo array con los valores que el Callback retorna.



map (2/3)

La función **map**, pasa tres parámetros al Callback:

1. El elemento actual del array
2. El índice en el array del elemento actual
3. El array completo que se está mapeando

Supongamos que tenemos una aplicación que mantiene un array de mascotas.

Cada mascota es un objeto JSON.

```
1  var mascotas =[
2      {
3          'nombre': 'Bobby',
4          'edad': 3
5      },
6      {
7          'nombre': 'Pancho',
8          'edad': 1
9      },
10     {
11         'nombre': 'Alfred',
12         'edad': 2
13     },
14
15 ];
```




map (3/3)

Digamos que queremos crear un nuevo array con el nombre de las mascotas solamente.

Usando **map** podemos escribir:

```
16
17
18
19   var nombres_Mascotas = mascotas.map(function(mascota, indice, array){
20       return mascota.nombre;
21   })
22
23
24
```

Los parámetros índice y array generalmente no se usan y se pueden omitir. El código seguiría funcionando normalmente.

El Callback que pasamos al map debe tener una sentencia return específica, o el map devolverá un array lleno de **undefined**.



Ejemplos



filter (1/2)

La función **filter**, toma un array y filtra los elementos no deseados.

Al igual que map, **filter** está disponible para cualquier array y se le pasa un Callback como primer parámetro.

filter ejecuta ese Callback para cada uno de los elementos del array y devuelve un nuevo array que contiene sólo los elementos para los que la devolución del Callback devuelve **true**.



filter (2/2)

filter, pasa tres parámetros al Callback:

1. El elemento actual del array
2. El índice en el array del elemento actual
3. El array que se está filtrando

Vamos a revisar nuestro ejemplo de mascotas.

En lugar de obtener los nombres de las mascotas, quiero las mascotas mayores a 1 año.

```
25  
26  
27   var MascotasMayoresA1 = mascotas.filter(function(mascota){  
28       return mascota.edad > 1;  
29   })  
30  
31
```

Con **filter** debemos asegurarnos que el Callback va a retornar un valor **booleano**.



Ejemplos



reduce (1/2)

map crea un nuevo array mediante la transformación de cada elemento de un array, de forma individual.

filter crea un nuevo array, eliminando los elementos que no le pertenecen.

reduce, por otro lado, toma todos los elementos de un array, y los reduce en un solo valor.

reduce, pasa cuatro parámetros al Callback:

1. El valor anterior
2. El valor actual
3. El índice en el array del elemento actual
4. El array que se está reduciendo



reduce (2/2)

Supongamos que quiero obtener la suma de las edades de las mascotas.

```
var sumaEdades = mascotas.reduce(function(anterior, actual){  
  return anterior + actual.edad;  
}, 0);
```

El segundo parámetro de reduce, es el valor inicial que tomará la función Callback para la primera iteración.



Ejemplos



Ejercitación